

Exercices — La récursivité

Aide-mémoire C++

Chaînes (`std::string`, bibliothèque `<string>`)

Outil	Rôle	Exemple avec <code>s = "Code"</code>
<code>s.length()</code> ou <code>s.size()</code>	Nombre de caractères	4
<code>s.empty()</code>	Vrai si la chaîne est vide	false
<code>s[i]</code>	Caractère à l'indice <code>i</code> (le premier est à l'indice 0)	<code>s[0] → 'C'</code>
<code>s[s.length() - 1]</code>	Dernier caractère	'e'
<code>s.substr(debut)</code>	Sous-chaîne de <code>debut</code> jusqu'à la fin	<code>s.substr(1) → "ode"</code>
<code>s.substr(debut, longueur)</code>	Sous-chaîne de <code>longueur</code> caractères à partir de <code>debut</code>	<code>s.substr(1, 2) → "od"</code>
<code>==</code> et <code>!=</code>	Comparer deux caractères ou deux chaînes	<code>s[0] == 'C' → true</code>
<code>+</code>	Concaténer deux <code>std::string</code> , ou une <code>std::string</code> et un <code>char</code>	<code>s + '!' → "Code!"</code>

Attention : le deuxième paramètre de `substr` est une **longueur**, pas un indice de fin. `s.substr(1, 2)` donne 2 caractères à partir de l'indice 1.

Tableaux

- Un tableau est passé à une fonction avec sa taille : `int somme(int tab[], int taille)`.
- Une fonction ne peut pas « raccourcir » un tableau, mais elle peut travailler sur **une partie** de celui-ci en modifiant la taille ou des indices passés en paramètres.

Niveau 0 — Lire avant d'écrire

Ces exercices se font **sur papier**, sans ordinateur. Vérifiez ensuite vos réponses en exécutant le code.

Exercice 0.1 — Prédire l'affichage

```
void mystere(int n)
{
    if (n > 0)
    {
        std::cout << n << " ";
    }
}
```

```
    mystere(n - 1);  
    std::cout << n << " ";  
  }  
}
```

- a) Qu'affiche `mystere(3)` ?
- b) Combien d'appels à `mystere` sont effectués au total ?
- c) Où se trouve le cas de base de cette fonction ?

Exercice 0.2 — Tracer la pile d'appels

```
int f(int a, int b)
{
    if (b == 0)
    {
        return 0;
    }
    return a + f(a, b - 1);
}
```

- a) Tracez l'exécution de $f(3, 4)$ en montrant la descente et la remontée, comme dans la théorie.
- b) Quelle opération mathématique cette fonction calcule-t-elle ?
- c) Que se passe-t-il avec $f(3, -2)$? Pourquoi ?

Exercice 0.3 — Trouver l'erreur

Chacune de ces fonctions contient une erreur. Pour chacune, expliquez le problème et proposez une correction.

a)

```
int factorielle(int n)
{
    return n * factorielle(n - 1);
}
```

b) Doit retourner $1 + 2 + \dots + n$.

```
int sommeJusqua(int n)
{
    if (n == 0)
    {
        return 0;
    }
    return n + sommeJusqua(n);
}
```

c) Doit retourner le nombre de chiffres de n ($n \geq 0$).

```
int compterChiffres(int n)
{
    if (n < 10)
    {
        return 1;
    }
    1 + compterChiffres(n / 10);
}
```

d) Doit retourner $n + (n - 2) + (n - 4) + \dots$ en s'arrêtant avant de devenir négatif. Elle fonctionne pour `sommeJusqua(6)`, mais pas pour `sommeJusqua(5)`. Pourquoi ?

```
int sommeJusqua(int n)
{
    if (n == 0)
    {
        return 0;
    }
    return n + sommeJusqua(n - 2);
}
```

Niveau 1 — Récursivité sur les nombres

Exercice 1.1 — Factorielle

La factorielle de n (notée $n!$) est le produit $1 \times 2 \times \dots \times n$. Par convention, $0! = 1$.

```
int factorielle(int n)    // n >= 0
```

Appel	Résultat attendu
<code>factorielle(0)</code>	1
<code>factorielle(1)</code>	1
<code>factorielle(5)</code>	120
<code>factorielle(10)</code>	3628800

- **Piste :** $5! = 5 \times 4!$

Exercice 1.2 — Triangle d'étoiles

a) Écrivez une fonction qui affiche n étoiles sur une même ligne (sans retour à la ligne).

```
void afficherLigne(int n)
```

`afficherLigne(3)` affiche `***`.

b) En utilisant `afficherLigne`, écrivez une fonction qui affiche un triangle de n lignes, **la plus courte en premier**.

```
void afficherTriangle(int n)
```

`afficherTriangle(4)` affiche :

```
*
**
***
****
```

c) Écrivez `afficherTriangleInverse(int n)`, qui affiche le triangle **la plus longue ligne en premier**.

- **Piste :** relisez la section « Sous le capot : la pile d'appels » de la théorie. Où faut-il placer l'affichage par rapport à l'appel récursif ?

Exercice 1.3 — Compter les chiffres

Écrivez une fonction qui retourne le nombre de chiffres d'un entier, positif ou négatif.

```
int compterChiffres(int n)
```

Appel	Résultat attendu
<code>compterChiffres(0)</code>	1
<code>compterChiffres(7)</code>	1
<code>compterChiffres(1024)</code>	4
<code>compterChiffres(-305)</code>	3

- **Piste :** en C++, `n / 10` retire le dernier chiffre d'un entier : `1024 / 10` vaut `102`, et `-305 / 10` vaut `-30`.

Exercice 1.4 — Somme des chiffres

Écrivez une fonction qui retourne la somme des chiffres d'un entier positif.

```
int sommeChiffres(int n)    // n >= 0
```

Appel	Résultat attendu
<code>sommeChiffres(0)</code>	0
<code>sommeChiffres(7)</code>	7
<code>sommeChiffres(1234)</code>	10
<code>sommeChiffres(9999)</code>	36

- **Piste :** `n % 10` donne le dernier chiffre (`1234 % 10` vaut 4).

Exercice 1.5 — PGCD (algorithme d'Euclide)

Le plus grand commun diviseur (PGCD) de deux entiers se calcule avec deux règles :

- `pgcd(a, 0) = a`
- `pgcd(a, b) = pgcd(b, a % b)`

```
int pgcd(int a, int b)    // a >= 0, b >= 0, pas tous les deux nuls
```

Appel	Résultat attendu
<code>pgcd(48, 18)</code>	6
<code>pgcd(18, 48)</code>	6
<code>pgcd(17, 5)</code>	1
<code>pgcd(7, 0)</code>	7
<code>pgcd(0, 9)</code>	9

Question : pourquoi est-on certain que cette fonction finit par s'arrêter ?